

Enterprise RAG Document Intelligence Platform

Project Documentation

Version 2.0

Technology Stack

Python | Streamlit | LangChain | FAISS | Hugging Face | Google Gemini | OpenAI | Docker |
Kubernetes | GitHub Actions | Git | PyPDF | Sentence Transformers

Table of Contents

1. Executive Summary
2. Introduction
3. Problem Statement
4. Existing System
5. Proposed System
6. Objectives
7. Features of the System
8. Technology Stack Overview
9. System Architecture
10. Document Processing Pipeline
11. Embedding & Vector Storage
12. Retrieval Engine
13. LLM Generation Layer
14. Streamlit Web Application
15. Docker Containerization
16. Kubernetes Deployment
17. CI/CD Pipeline
18. Security Considerations
19. Testing Strategy
20. Project Structure
21. Future Enhancements
22. Conclusion
23. References

1. Executive Summary

The Enterprise RAG Document Intelligence Platform is an AI-powered document question-answering system that enables users to interact with large collections of PDF documents using natural language. The application leverages Retrieval-Augmented Generation (RAG), which combines semantic document retrieval with Large Language Models (LLMs) to generate accurate, context-aware responses.

Unlike traditional chatbots that rely solely on pretrained knowledge, this platform retrieves relevant information directly from uploaded documents before generating responses. This significantly reduces hallucinations and improves factual accuracy.

The application is built using Python and Streamlit, providing an intuitive web interface where users can upload documents and ask questions. LangChain orchestrates the document processing workflow, while Hugging Face sentence-transformer models generate dense vector embeddings. These embeddings are stored in a FAISS vector database for efficient semantic search.

When a user submits a query, the system retrieves the most relevant document chunks using cosine similarity search. The retrieved context is then passed to Google Gemini or OpenAI models to generate precise and contextually relevant answers.

To support enterprise deployment, the application is containerized using Docker and designed for orchestration with Kubernetes. Automated build, testing, and deployment processes are implemented using GitHub Actions, enabling a complete CI/CD pipeline.

This project demonstrates modern Generative AI architecture and enterprise deployment practices, making it suitable for intelligent document management, knowledge retrieval, customer support, legal document analysis, research assistance, financial reporting, and enterprise knowledge bases.

2. Introduction

Organizations today generate massive volumes of digital documents including reports, technical manuals, research papers, policies, contracts, invoices, compliance documents, and operational procedures. Locating specific information within these documents often requires extensive manual effort, reducing productivity and increasing operational costs.

Traditional keyword-based search engines frequently fail to understand semantic relationships between user queries and document content. Users are forced to manually browse multiple documents before locating the required information.

Recent advancements in Large Language Models (LLMs) have transformed natural language understanding and generation. However, LLMs possess a static knowledge base and cannot access organization-specific information unless explicitly provided.

Retrieval-Augmented Generation (RAG) solves this limitation by combining semantic retrieval with generative AI. Instead of relying only on model knowledge, the system retrieves relevant document sections before generating answers. This approach improves factual accuracy, minimizes hallucinations, and enables AI systems to answer questions based solely on trusted organizational documents.

The Enterprise RAG Document Intelligence Platform was developed to provide an intelligent document assistant capable of understanding user questions, retrieving relevant information from uploaded documents, and generating accurate responses in real time.

3. Problem Statement

Organizations often store thousands of PDF documents containing valuable business knowledge. Retrieving relevant information from these documents is time-consuming and inefficient.

Existing document management systems primarily rely on keyword matching, which cannot understand semantic meaning or contextual relationships. Consequently, users spend considerable time manually searching documents.

Furthermore, conventional AI chatbots cannot answer questions related to organization-specific documents because such information is absent from their training datasets.

The challenge addressed by this project is to design an enterprise-grade intelligent document assistant capable of understanding user intent, retrieving semantically relevant content, and generating trustworthy answers grounded in uploaded documents.

4. Existing System

Traditional document search systems depend on keyword matching techniques. These systems require users to know the exact terms present in documents. Minor wording differences often produce poor search results.

Conventional search solutions also lack contextual understanding, making it difficult to answer natural language questions such as:

- What are the key responsibilities of the finance department?
- Summarize the risk management strategy.
- What were the revenue figures mentioned in the annual report?

Similarly, standalone Large Language Models cannot retrieve confidential or organization-specific documents without additional retrieval mechanisms.

Limitations of Existing Systems

- Keyword-based search only
- No semantic understanding
- Manual document browsing
- High search time
- Increased operational effort
- Hallucinations in generic AI chatbots
- No enterprise document integration
- Poor scalability for large document collections

5. Proposed System

The proposed system introduces a Retrieval-Augmented Generation (RAG) architecture capable of performing intelligent document retrieval and answer generation.

The application begins by accepting PDF uploads through a Streamlit interface. Documents are parsed using PyPDF and divided into manageable text chunks using LangChain's RecursiveCharacterTextSplitter.

Each text chunk is converted into dense vector embeddings using Hugging Face sentence-transformer models. These embeddings are stored inside a FAISS vector database, enabling efficient semantic similarity search.

When a user submits a question, the application generates an embedding for the query and retrieves the most relevant document chunks from FAISS. The retrieved context is then

combined with the user's question and sent to Google Gemini or OpenAI to produce a context-aware answer.

This architecture ensures that responses are generated using retrieved document information rather than relying solely on the LLM's pretrained knowledge.

6. Objectives

The primary objectives of this project are:

- Develop an intelligent document question-answering platform.
- Implement Retrieval-Augmented Generation (RAG) architecture.
- Perform semantic document retrieval using FAISS.
- Generate accurate embeddings using Hugging Face models.
- Build an interactive web application using Streamlit.
- Integrate Google Gemini and OpenAI for natural language generation.
- Deploy the application using Docker containers.
- Orchestrate deployment using Kubernetes.
- Automate build and deployment using GitHub Actions CI/CD.
- Design an enterprise-ready scalable architecture.
- Improve search accuracy through semantic retrieval.
- Reduce hallucinations by grounding responses in uploaded documents.
- Provide an intuitive and responsive user experience.

7. System Architecture

The Enterprise RAG Document Intelligence Platform follows a modular and scalable architecture based on the Retrieval-Augmented Generation (RAG) paradigm. Each module is designed as an independent component, allowing easier maintenance, testing, deployment, and future enhancements. The architecture separates document processing, semantic retrieval, language model interaction, and user interface into distinct layers, ensuring loose coupling and high scalability.

The application begins when a user uploads one or more PDF documents through the Streamlit interface. Uploaded files are processed by the document ingestion module, which extracts textual content and prepares it for semantic indexing. The extracted text is divided into overlapping chunks using LangChain's RecursiveCharacterTextSplitter, ensuring that contextual information is preserved across chunk boundaries.

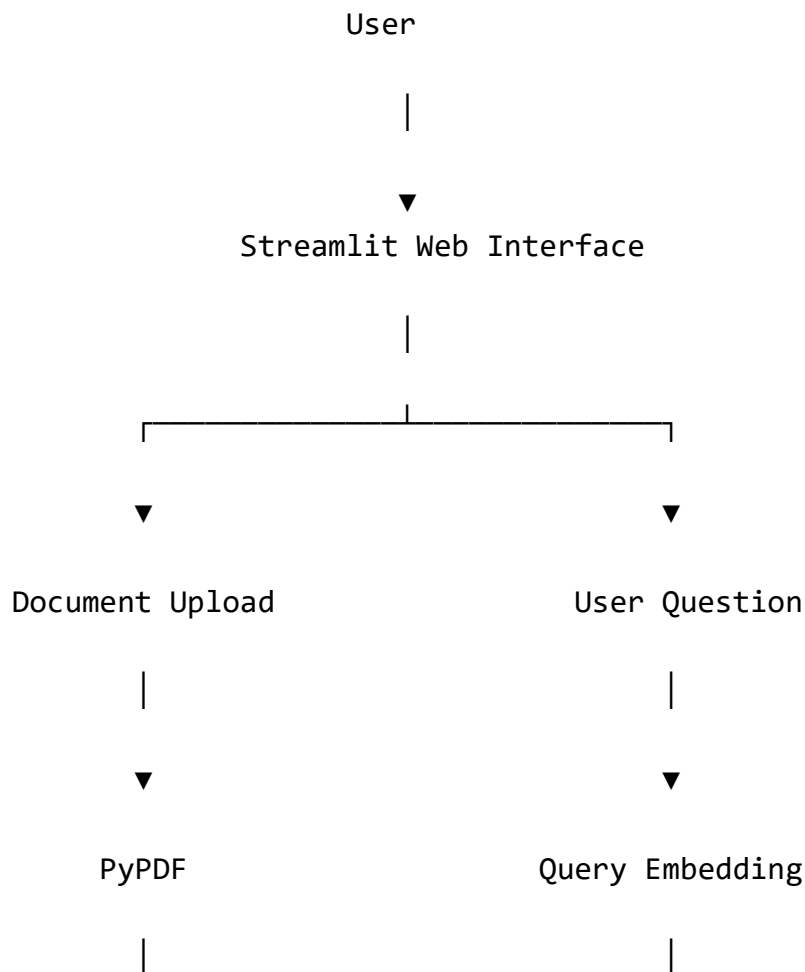
Each text chunk is converted into a dense numerical vector using Hugging Face Sentence Transformer embeddings. These embeddings are stored in a FAISS vector database, enabling extremely fast similarity searches over large document collections.

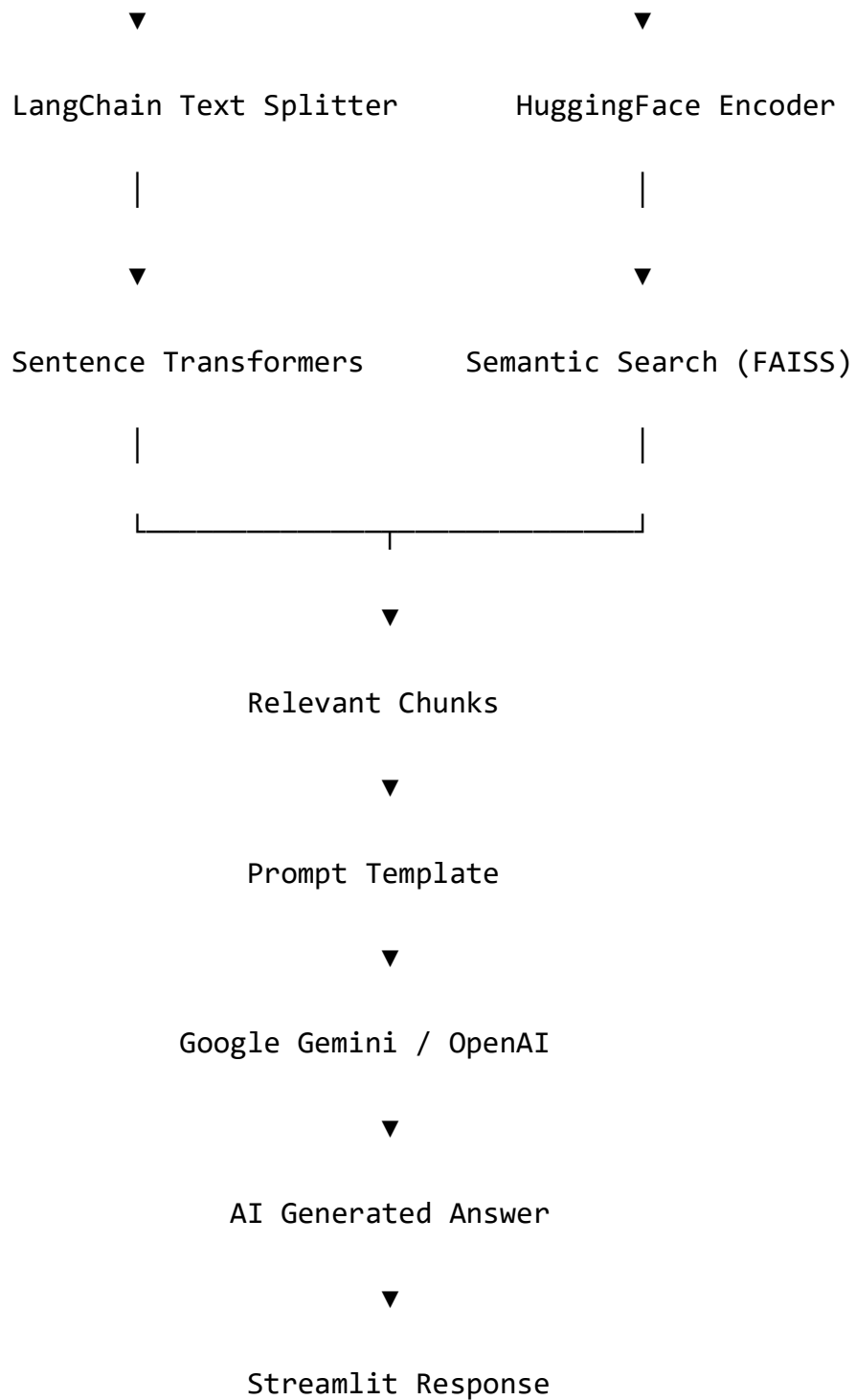
When a user submits a question, the same embedding model converts the query into a vector representation. FAISS compares the query vector against all indexed document vectors and retrieves the most semantically relevant chunks.

The retrieved context is combined with the user's question through a carefully designed prompt template. This prompt is forwarded to Google Gemini or OpenAI, which generates a context-aware response grounded in the retrieved document content.

Finally, the generated answer is displayed within the Streamlit chat interface while maintaining conversation history for future interactions.

Overall Architecture





The modular architecture ensures each component performs a dedicated responsibility, improving maintainability and allowing independent upgrades without affecting the overall system.

8. Document Processing Pipeline

The document processing pipeline is responsible for converting uploaded PDF documents into semantically searchable knowledge.

The pipeline consists of multiple sequential stages that transform raw document data into indexed vector representations suitable for semantic retrieval.

Stage 1 – Document Upload

Users upload one or more PDF documents through the Streamlit web application. Uploaded files are temporarily stored and validated before processing begins.

Validation checks include:

- Supported file format
- File size
- Empty document detection
- Corrupted file verification

Only valid documents proceed to the ingestion stage.

Stage 2 – PDF Parsing

The uploaded document is parsed using the PyPDF library.

PyPDF extracts textual information from every page while preserving reading order.

The parser handles:

- Multi-page PDFs
- Paragraph extraction
- Header detection
- Footer removal
- Page numbering
- Empty page filtering

The extracted text becomes the input for preprocessing.

Stage 3 – Text Preprocessing

Before indexing, extracted text undergoes preprocessing to improve embedding quality.

Operations include:

- Unicode normalization
- Whitespace removal
- Empty line elimination
- Special character cleanup
- Page separator handling
- Text formatting

These operations reduce unnecessary noise and improve retrieval accuracy.

Stage 4 – Text Chunking

Large Language Models have token limitations.

Instead of embedding an entire document, LangChain's RecursiveCharacterTextSplitter divides the document into overlapping chunks.

Typical configuration:

- Chunk Size : 1000 characters
- Chunk Overlap : 200 characters

The overlap preserves contextual continuity between adjacent chunks.

Each chunk receives metadata including:

- Document Name
- Page Number
- Chunk ID
- Source File
- Chunk Index

This metadata helps generate accurate citations during retrieval.

Stage 5 – Embedding Generation

Each chunk is converted into a dense vector using Hugging Face Sentence Transformers.

Example model:

sentence-transformers/all-MiniLM-L6-v2

or

all-mpnet-base-v2

The embedding captures semantic meaning rather than simple keywords.

Semantically similar sentences produce vectors that are positioned close together within the embedding space.

Stage 6 – Vector Storage

Generated embeddings are stored inside FAISS.

Each vector is stored together with:

- Chunk Text
- Metadata
- Document Name
- Page Number
- Chunk Index

This allows rapid retrieval while maintaining traceability to the original source document.

9. LangChain Workflow

LangChain serves as the orchestration framework that connects every stage of the Retrieval-Augmented Generation pipeline.

Instead of manually implementing each processing step, LangChain provides reusable components that simplify document loading, chunking, retrieval, prompting, and response generation.

The workflow consists of several interconnected modules.

Document Loader

The Document Loader reads uploaded PDF files and converts them into LangChain Document objects.

Each document contains:

- Page Content
- Metadata
- Source Information

These objects become the foundation for subsequent processing.

Recursive Character Text Splitter

The RecursiveCharacterTextSplitter divides large documents into manageable chunks while maintaining contextual continuity.

Unlike fixed-length splitting, recursive splitting intelligently separates content using:

- Paragraphs
- Sentences
- Line breaks
- Spaces

This significantly improves semantic retrieval accuracy.

Embedding Model

LangChain integrates directly with Hugging Face embedding models.

Each document chunk is transformed into a high-dimensional vector representation suitable for semantic search.

Embedding generation occurs only once during document indexing.

Vector Store Integration

LangChain provides seamless integration with FAISS.

The framework automatically:

- Creates the FAISS index
- Stores vectors
- Saves metadata
- Performs similarity searches

This abstraction significantly reduces development complexity.

Retriever

The Retriever accepts a user query and returns the most relevant document chunks.

Typical configuration:

- Top K Results = 5
- Similarity Search
- Metadata Filtering
- Score Ranking

Only highly relevant chunks are forwarded to the language model.

Prompt Template

LangChain constructs structured prompts before invoking the LLM.

The prompt contains:

- System Instructions
- Retrieved Context
- User Question

Example:

You are an AI Document Assistant.

Answer only from the provided context.

If the answer is unavailable,
reply that sufficient information was
not found.

Prompt engineering plays an important role in reducing hallucinations.

LLM Chain

The prompt is forwarded to:

- Google Gemini

or

- OpenAI GPT

The selected language model generates a natural language response using only the retrieved context.

10. Embedding and Vector Storage

Semantic search is the foundation of Retrieval-Augmented Generation.

Instead of matching keywords, the system compares numerical vector representations of text.

This enables the application to understand meaning rather than exact wording.

Hugging Face Sentence Transformers

Sentence Transformers convert sentences into dense vectors.

Advantages include:

- Semantic understanding
- Fast inference
- Local execution
- High retrieval accuracy
- No dependency on keyword matching

Each document chunk is transformed into a fixed-length embedding.

FAISS Vector Database

Facebook AI Similarity Search (FAISS) is an optimized vector database designed for high-speed similarity search.

Major responsibilities include:

- Store embeddings
- Build vector indexes
- Perform nearest-neighbor search
- Retrieve Top-K results
- Scale to millions of vectors

Compared to traditional databases, FAISS provides extremely low-latency retrieval.

Similarity Search

When a query is submitted:

1. Generate query embedding.

2. Compare against indexed document vectors.
3. Compute cosine similarity.
4. Rank results.
5. Return the Top-K document chunks.

This allows users to ask questions using natural language instead of exact keywords.

11. Retrieval Engine

The Retrieval Engine acts as the bridge between the indexed knowledge base and the Large Language Model.

Its responsibilities include:

- Query embedding generation
- Semantic similarity search
- Ranking retrieved chunks
- Metadata retrieval
- Context preparation

The retrieval process ensures that only relevant document information is passed to the language model.

This greatly improves factual accuracy while minimizing hallucinations.

The retrieved document chunks are formatted with source information before being forwarded to the prompt generation module, enabling traceable and trustworthy AI responses.

12. LLM Generation Layer

The Large Language Model (LLM) Generation Layer is responsible for transforming retrieved document context into accurate, human-readable responses. Unlike traditional AI chatbots that rely solely on pretrained knowledge, the Enterprise RAG Document Intelligence Platform first retrieves relevant document content before invoking the language model. This Retrieval-Augmented Generation (RAG) approach significantly improves response accuracy while reducing hallucinations.

The application supports multiple LLM providers, allowing organizations to choose the model that best suits their performance, cost, and deployment requirements.

Supported models include:

- Google Gemini 2.5 Flash / Gemini 2.5 Pro
- OpenAI GPT-4o / GPT-4.1 (optional)
- Future support for other enterprise LLMs

This modular design ensures that switching between providers requires minimal configuration changes.

Prompt Engineering

Prompt engineering plays a critical role in controlling the behavior of the language model.

Instead of sending only the user's question, the application constructs a structured prompt containing:

- System Instructions
- Retrieved Document Context
- Conversation History (Optional)
- User Query

Example Prompt Structure:

System:

You are an Enterprise AI Document Assistant.

Answer only using the provided document context.

If the information is unavailable, respond that the document does not contain sufficient information.

Context:

<Document Chunks Retrieved from FAISS>

Question:

<User Question>

This approach ensures that generated responses remain grounded in the uploaded documents.

Response Generation Workflow

The response generation process follows these steps:

1. User submits a question.

2. Query embedding is generated.
3. FAISS retrieves the most relevant document chunks.
4. LangChain constructs the final prompt.
5. Google Gemini or OpenAI processes the prompt.
6. The generated answer is returned to Streamlit.
7. Conversation history is updated.

This workflow minimizes latency while ensuring contextual accuracy.

Advantages of Using Google Gemini

Google Gemini offers several advantages for enterprise document intelligence systems:

- Advanced reasoning capabilities
- Long context window
- Fast response generation
- High factual consistency
- Efficient handling of large documents
- Strong multilingual support
- Integration with LangChain

When OpenAI is configured, the workflow remains identical, allowing organizations to switch providers without major architectural changes.

13. Streamlit Web Application

The front-end of the application is developed using Streamlit, providing a lightweight yet powerful interface for interacting with enterprise documents.

Streamlit enables rapid deployment while offering an intuitive user experience without requiring frontend frameworks.

The interface is divided into two primary sections:

- Sidebar (Document Management)
- Main Chat Window

Document Upload Panel

The sidebar allows users to upload one or multiple PDF documents.

Features include:

- Multiple PDF upload support
- Drag-and-drop interface
- Upload progress indicator
- File validation
- Duplicate file detection
- Document processing status

After upload, the application automatically begins document parsing and indexing.

Chat Interface

The central chat window provides a conversational interface for querying uploaded documents.

Features include:

- User message bubbles
- AI response bubbles
- Conversation history
- Scrollable interface
- Real-time response rendering
- Citation-ready responses

The interface is designed to resemble modern AI assistants, improving usability for enterprise users.

Session Management

Streamlit Session State is used to preserve application state during user interaction.

Stored information includes:

- Uploaded document list
- FAISS index reference
- Conversation history
- User queries
- Generated responses

This prevents unnecessary reprocessing of documents during a session.

Error Handling

Robust error handling improves user experience by preventing unexpected failures.

Handled scenarios include:

- Missing API keys
- Invalid PDF files
- Empty documents
- Unsupported file types
- Network failures
- API rate limits
- Empty retrieval results

Meaningful error messages are displayed to guide the user.

14. Configuration and Environment

Application configuration is managed using environment variables stored in a `.env` file.

This approach separates sensitive credentials from application code and simplifies deployment across multiple environments.

Typical environment variables include:

```
GOOGLE_API_KEY=xxxxxxxxxxxxx
```

```
OPENAI_API_KEY=xxxxxxxxxxxxx
```

```
EMBEDDING_MODEL=sentence-transformers/all-MiniLM-L6-v2
```

```
VECTOR_STORE=FAISS
```

```
CHUNK_SIZE=1000
```

```
CHUNK_OVERLAP=200
```

```
TOP_K_RESULTS=5
```

```
STREAMLIT_SERVER_PORT=8501
```

Using environment variables improves security and enables easy configuration changes without modifying source code.

15. Docker Containerization

Docker enables the application to run consistently across development, testing, and production environments by packaging the application and all its dependencies into lightweight containers.

Containerization eliminates environment-specific issues and simplifies deployment.

Why Docker?

Without Docker, applications often encounter dependency conflicts across different systems.

Docker solves this by packaging:

- Python Runtime
- Required Libraries
- Streamlit
- LangChain
- FAISS
- Hugging Face Models
- Application Source Code

into a single portable image.

Multi-Stage Docker Build

The project uses a multi-stage Dockerfile to reduce image size and improve build performance.

Builder Stage

Responsibilities include:

- Install Python dependencies
- Download required packages
- Build application environment

Runtime Stage

Only the necessary runtime files are copied from the builder stage.

Benefits include:

- Smaller image size
- Faster deployment
- Improved security
- Reduced attack surface

Docker Architecture

Developer

|



Docker Build

|



Docker Image

|



Docker Container

|



Streamlit Application

|



Google Gemini / OpenAI

Dockerfile Overview

The Dockerfile performs the following tasks:

- Uses Python base image
- Creates working directory
- Copies application source code
- Installs dependencies
- Exposes Streamlit port
- Defines startup command

Example startup command:

```
streamlit run app.py
```

Docker Compose

Docker Compose simplifies local development by defining all required services in a single YAML file.

The Compose configuration includes:

- Streamlit Application
- Persistent Volume
- Environment Variables
- Restart Policy
- Network Configuration

Benefits include:

- One-command deployment
- Simplified development
- Consistent environments
- Easy configuration management

Container Lifecycle

Docker manages the application lifecycle through several stages:

1. Build Image
2. Create Container
3. Start Container
4. Execute Application
5. Stop Container
6. Remove Container

Each stage ensures predictable application behavior across environments.

Docker Volumes

Persistent Docker volumes are used to retain application data even when containers are recreated.

Typical stored data includes:

- Uploaded PDFs
- FAISS Vector Index
- Logs
- Cached Files

This ensures document embeddings remain available across container restarts.

Health Checks

Docker periodically verifies application health using HTTP health endpoints.

Typical health checks include:

- Streamlit server availability
- API responsiveness
- Container readiness

If the application becomes unhealthy, Docker can automatically restart the container depending on the configured restart policy.

Benefits of Docker

Using Docker provides several enterprise advantages:

- Platform independence
- Faster deployments
- Simplified dependency management
- Consistent runtime environment
- Easier scaling
- Improved portability
- Better resource utilization
- Simplified DevOps integration

Docker serves as the foundation for deploying the application on Kubernetes, enabling enterprise-grade scalability and reliability.

16. Kubernetes Deployment

Overview

As enterprise applications scale, managing multiple Docker containers manually becomes increasingly difficult. Kubernetes is an open-source container orchestration platform that automates deployment, scaling, networking, monitoring, and recovery of containerized applications.

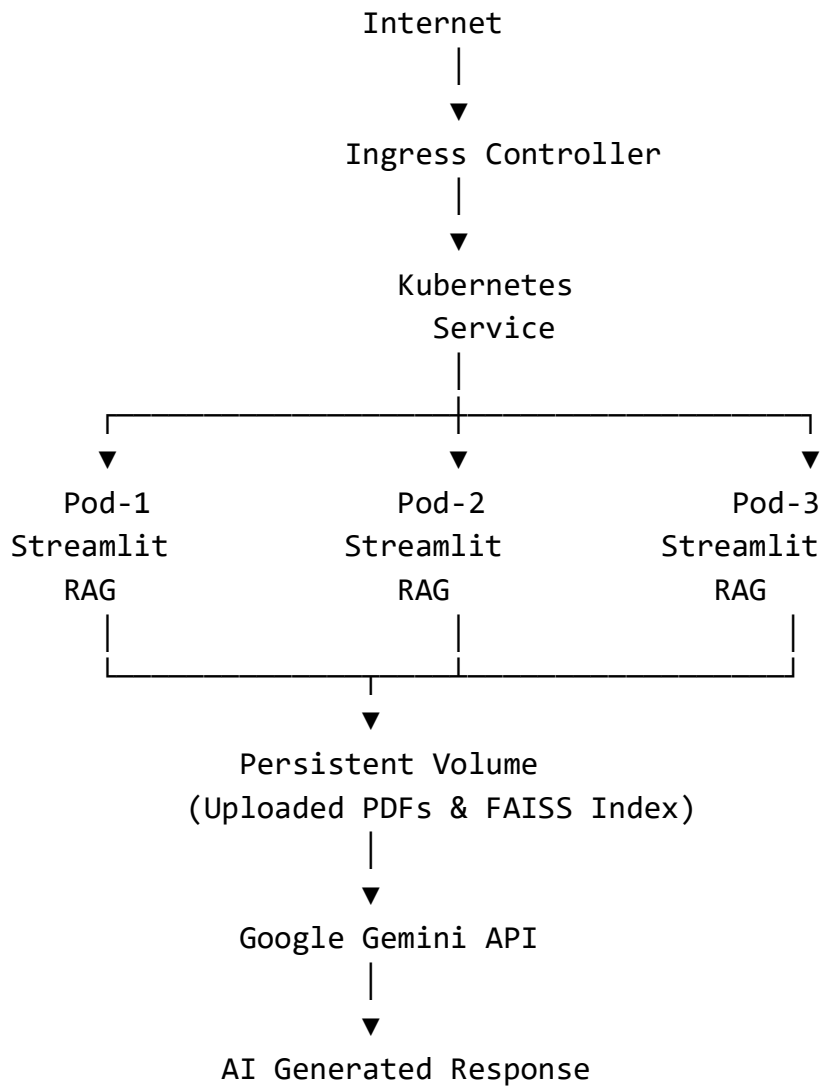
The Enterprise RAG Document Intelligence Platform is designed to be Kubernetes-ready, enabling seamless deployment across cloud environments such as AWS EKS, Azure AKS, Google GKE, or on-premise Kubernetes clusters.

Kubernetes provides several enterprise features including:

- Automated deployment
- Load balancing
- Horizontal scaling
- Self-healing
- Rolling updates
- Resource management
- Service discovery
- High availability

These capabilities ensure that the RAG application remains reliable even under heavy workloads.

Kubernetes Architecture



Kubernetes Components

Namespace

The application resources are isolated inside a dedicated namespace.

Benefits:

- Better organization
- Resource isolation
- Easier monitoring
- Simplified management

Deployment

The Deployment resource manages application Pods.

Responsibilities include:

- Creating Pods
- Rolling Updates
- Replica Management
- Pod Recovery

Typical configuration:

- 3 Replicas
- Rolling Update Strategy
- Automatic Restart
- Resource Requests and Limits

Service

The Kubernetes Service exposes the application inside the cluster.

Responsibilities:

- Load balancing
- Service discovery
- Stable network endpoint
- Internal communication

ConfigMap

Configuration values such as chunk size, embedding model, and retrieval settings are stored in ConfigMaps.

Advantages:

- Centralized configuration
- Easy updates
- Environment separation
- Reduced hardcoding

Secrets

Sensitive information including API keys is stored securely using Kubernetes Secrets.

Examples:

- GOOGLE_API_KEY
- OPENAI_API_KEY

Benefits:

- Secure storage
- Environment isolation
- Encryption support
- Better compliance

Persistent Volume

Persistent Volumes ensure uploaded PDFs and FAISS indexes remain available even if Pods restart.

Stored data includes:

- Uploaded documents
- Vector database
- Application logs
- Cache

Horizontal Pod Autoscaler (HPA)

The HPA automatically increases or decreases application replicas depending on workload.

Example configuration:

Minimum Pods : 2

Maximum Pods : 10

CPU Threshold : 70%

Memory Threshold : 80%

This improves application availability during peak usage.

Rolling Updates

Rolling Updates enable zero-downtime deployment.

During deployment:

- New Pods start first.
- Old Pods terminate gradually.
- Users experience uninterrupted service.

Self-Healing

Kubernetes continuously monitors Pod health.

If a Pod fails:

- Kubernetes removes the unhealthy Pod.
- A new Pod is created automatically.
- Application availability is maintained.

Benefits of Kubernetes

- Automatic Scaling
- High Availability
- Fault Tolerance
- Self Healing
- Zero Downtime Deployment
- Better Resource Utilization
- Enterprise Ready Infrastructure

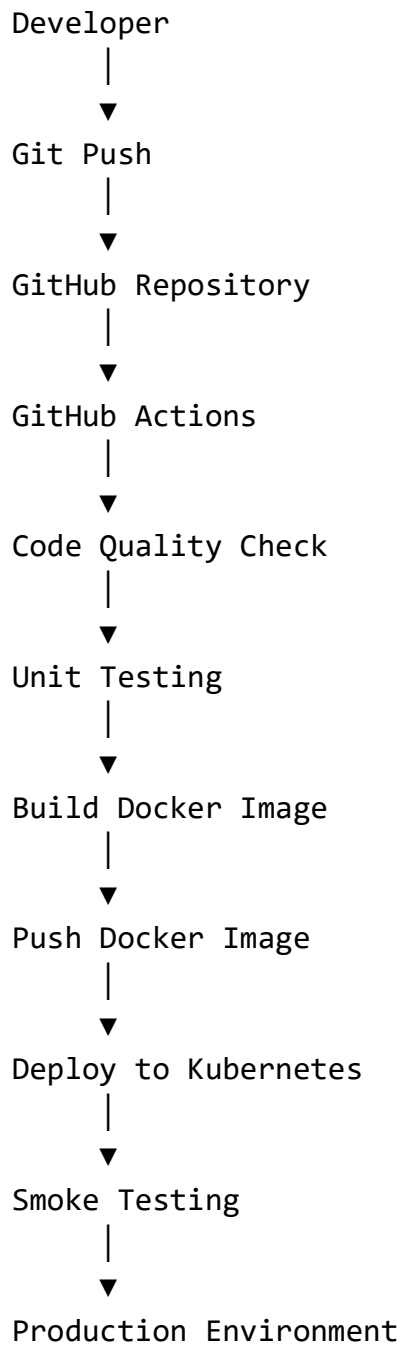
17. CI/CD Pipeline

Introduction

Continuous Integration and Continuous Deployment (CI/CD) automate software delivery by reducing manual deployment effort and ensuring consistent software quality.

The Enterprise RAG Document Intelligence Platform uses GitHub Actions to automate testing, Docker image creation, and Kubernetes deployment.

CI/CD Workflow



Pipeline Stages

Stage 1 – Source Code

Developers push code to GitHub.

Every push automatically triggers the pipeline.

Stage 2 – Static Analysis

The workflow verifies:

- Code formatting
- Syntax
- Dependency validation
- Linting

Stage 3 – Unit Testing

Automated tests validate:

- Document Processing
- Embedding Generation
- Retrieval Logic
- API Integration

Only successful builds continue.

Stage 4 – Docker Build

GitHub Actions builds the Docker image.

Benefits:

- Repeatable builds
- Version tagging
- Faster deployment

Stage 5 – Image Registry

Built Docker images are uploaded to:

- Docker Hub

or

- GitHub Container Registry

or

- Amazon ECR

Stage 6 – Kubernetes Deployment

The workflow deploys the latest image using:

- kubectl
- Kubernetes manifests

Deployment includes:

- Rolling updates
- Health verification
- Replica synchronization

Stage 7 – Smoke Testing

Basic tests verify:

- Application availability
- Streamlit startup
- API connectivity
- LLM communication

Advantages of CI/CD

- Faster Deployment
- Reduced Human Errors
- Automated Testing
- Consistent Builds
- Easier Rollback
- Continuous Delivery
- Enterprise DevOps Support

18. Security Considerations

Security is a critical aspect of enterprise AI applications.

The platform implements multiple layers of protection.

API Security

API keys are never hardcoded.

Instead they are stored using:

- Environment Variables
- Kubernetes Secrets

Secure File Upload

Uploaded documents undergo validation.

Checks include:

- File Extension
- File Size
- Empty Files
- Corrupted PDFs

Prompt Injection Protection

The application instructs the LLM to answer only from retrieved context.

This minimizes:

- Hallucinations
- Prompt Injection
- Irrelevant Responses

Container Security

Docker containers follow best practices:

- Non-root User
- Minimal Base Image
- Updated Dependencies
- Environment Isolation

Kubernetes Security

Security measures include:

- Namespace Isolation
- Secret Management
- Role-Based Access Control (RBAC)
- Network Policies
- Resource Limits

19. Testing Strategy

Testing ensures reliability, accuracy, and maintainability.

The project includes multiple testing approaches.

Unit Testing

Tests individual components.

Examples:

- PDF Parsing
- Chunk Generation
- Embedding Creation
- FAISS Retrieval

Integration Testing

Verifies interaction between modules.

Examples:

- Streamlit → LangChain
- LangChain → FAISS
- FAISS → Gemini

Functional Testing

Validates complete application behavior.

Scenarios include:

- Upload PDF
- Ask Questions

- Generate Responses
- Display Results

Performance Testing

Measures:

- Response Time
- Retrieval Speed
- Memory Usage
- CPU Utilization

User Acceptance Testing

End users verify:

- Accuracy
- Usability
- Performance
- Stability

20. Project Structure

enterprise-rag/

|

├─ app.py

├─ requirements.txt

├─ .env

├─ Dockerfile

├─ docker-compose.yml

├─ kubernetes/

| └─ deployment.yaml

| └─ service.yaml

| └─ ingress.yaml

| └─ configmap.yaml

| └─ secret.yaml

| └─ pvc.yaml

|

└─ src/

| └─ loader.py

| └─ splitter.py

| └─ embeddings.py

| └─ vectorstore.py

| └─ retriever.py

| └─ llm.py

| └─ prompt.py

|

└─ uploads/

└─ faiss_index/

└─ static/

21. Future Enhancements

Future improvements include:

- OCR for scanned PDFs
- Image-based document understanding
- Multi-language support
- Voice-enabled document search
- User Authentication
- Role-Based Access Control
- Cloud Database Integration
- Elasticsearch Support
- Hybrid Search
- Real-time Monitoring
- Analytics Dashboard
- Multi-user Collaboration
- Chat History Persistence
- Vector Database Migration (Pinecone, Weaviate, Milvus)
- AI Agent Integration

22. Conclusion

The Enterprise RAG Document Intelligence Platform demonstrates how Retrieval-Augmented Generation can significantly improve enterprise document search by combining semantic retrieval with Large Language Models.

By integrating LangChain, Hugging Face embeddings, FAISS, Google Gemini/OpenAI, and Streamlit, the application provides accurate, context-aware responses grounded in uploaded documents.

Containerization using Docker and orchestration through Kubernetes enable scalable deployment, while GitHub Actions automates testing and deployment through a modern CI/CD pipeline.

The modular architecture, secure deployment strategy, and scalable design make the platform suitable for real-world enterprise environments, including legal firms, financial institutions, healthcare organizations, research centers, and corporate knowledge management systems.

23. References

1. LangChain Official Documentation
<https://python.langchain.com>
2. Streamlit Documentation
<https://docs.streamlit.io>
3. FAISS Documentation
<https://faiss.ai>
4. Hugging Face Documentation
<https://huggingface.co/docs>
5. Google Gemini API Documentation
<https://ai.google.dev>
6. OpenAI API Documentation
<https://platform.openai.com/docs>
7. Docker Documentation
<https://docs.docker.com>
8. Kubernetes Documentation
<https://kubernetes.io/docs>
9. GitHub Actions Documentation
<https://docs.github.com/actions>
10. PyPDF Documentation
<https://pypdf.readthedocs.io>